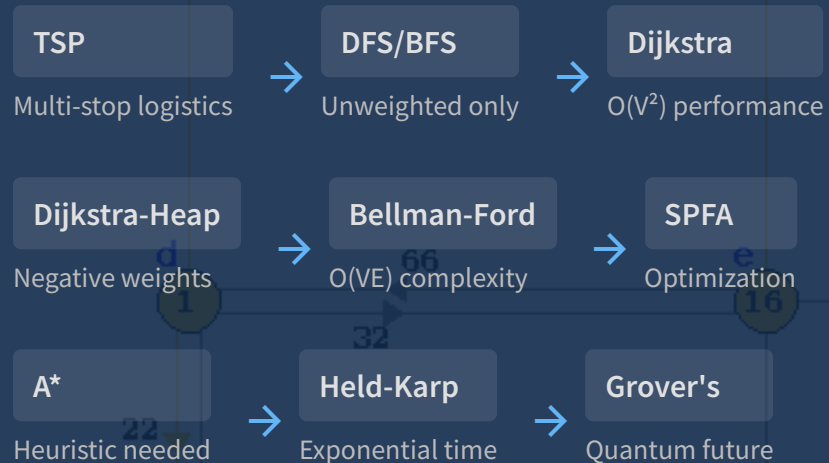


Algorithm Design and Implementation

Optimization and Advanced Theoretical Analysis (Phase 3)

From Multi-Stop Logistics to Algorithmic Frontiers

Our Algorithm Journey



Course: Algorithm (First Semester 2025/2026)

Instructor: Dr. Mohammed Abuhelaleh

Team Members:

Joud Kayyali - 3230601030

Performance

$O(V^2) \rightarrow$
 $O((V+E)\log V)$

Algorithms

**Dijkstra &
Bellman-Ford**

Challenge

**Negative
Weights**

Solution

Min-Heap & DP

Algorithm Evolution: Failures and Solutions



TSP Problem

1

- ❗ Multi-stop logistics with return to origin
- ❗ Exponential time complexity

→ Break into simpler subproblems

NP-Hard



DFS/BFS

2

- ❗ Only works for unweighted graphs
- ❗ Can't handle weighted edges

→ Need weighted graph algorithm

$O(V+E)$



Dijkstra (Set)

3

- ❗ $O(V^2)$ performance with large graphs
- ❗ Linear scan for minimum node

→ Use priority queue (Min-Heap)

$O(V^2)$



Dijkstra (MinHeap)

4

- ❗ Fails with negative edge weights
- ❗ Energy rebates break greedy assumption

→ Use dynamic programming approach

$O((V+E)\log V)$



Bellman-Ford

5

- ❗ $O(VE)$ complexity
- ❗ Slow for dense graphs

→ Optimize with SPFA

$O(VE)$



SPFA

6

- ❗ Still slow for dense graphs
- ❗ Worst-case $O(VE)$

→ Add heuristic guidance (A^*)

$O(kE)$

*Queue-based optimization of Bellman-Ford
Only relaxes nodes that have changed*



A^*

7

- ❗ Requires good heuristic
- ❗ Fails without accurate estimate

→ Use approximation for NP-Hard

$O(b^d)$



Held-Karp/Genetic

8

- ❗ Held-Karp: $O(n^2 \cdot 2^n)$ time
- ❗ Genetic: No optimality guarantee

→ Quantum approach (Grover's)

Exponential



Grover's Algorithm

9

- ❗ Requires quantum hardware
- ❗ Still exponential ($O(\sqrt{N})$)

→ Future paradigm needed

$O(\sqrt{N})$

Introduction and Background

1

Problem Definition & Algorithm Design

Core Problem

Optimal Delivery Route

Model

Weighted Directed Graph

Algorithm

Dijkstra's Algorithm

2

Implementation & Testing

Implementation

Linear Search Dijkstra

Complexity

$O(V^2)$

Limitation

No Negative Weights

3

Optimization & Advanced Analysis

Performance

Min-Heap Optimization

New Algorithm

Bellman-Ford

Objective

Handle Energy Rebates

Optimization 1: Min-Heap Priority Queue

↔ Linear Set vs. Binary Heap

Linear Set (Phase 2)

Access: $O(V)$

Insert: $O(1)$

Remove: $O(V)$

Binary Heap (Phase 3)

Access: $O(1)$

Insert: $O(\log V)$

Remove: $O(\log V)$

↔ Implementation in JavaScript

Array-Based Tree Representation

Parent: $\text{floor}[(i-1)/2]$

Left Child: $2i + 1$

Right Child: $2i + 2$

Key Method: `bubbleUp()` & `sinkDown()`



Performance Improvement

$O(V^2) \rightarrow O((V+E)\log V)$

Significant speedup for sparse graphs

Optimization 2: Bellman-Ford Algorithm

! Why Bellman-Ford?

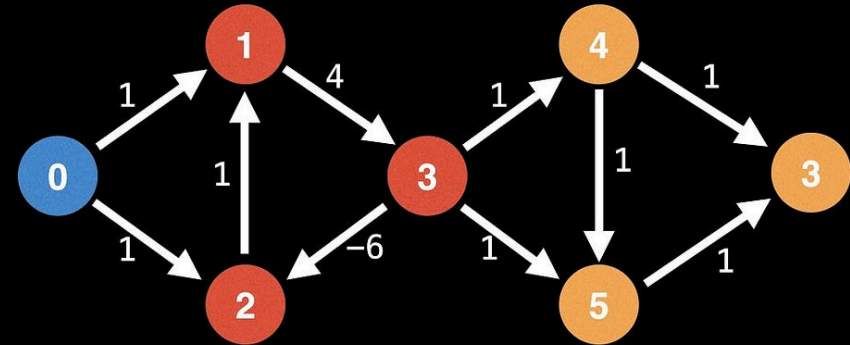
Dijkstra's Limitation

- ❗ Fails with **negative edge weights** (Energy Rebates)
- ❗ Greedy assumption: once visited, path is final
- ❗ Cannot detect negative cycles (infinite energy)

<> Implementation Details

- 🔄 **Relaxation** over $|V| - 1$ iterations
- 🔄 Dynamic Programming approach vs. Greedy
- ⚠️ **Negative Cycle Detection** in iteration $|V|$
- 🔗 Edge List representation (not Adjacency List)

Bellman-Ford Shortest Path Algorithm



Dijkstra

Approach: **Greedy**

Complexity: $O((V+E)\log V)$

Bellman-Ford

Approach: **Dynamic**

Complexity: $O(V \cdot E)$

Energy Grid Solution

Handles Negative Weights

Correctness over Speed

Complexity Analysis

Phase 3 introduces significant algorithmic improvements, with different trade-offs for specific use cases

Algorithm	Data Structure	Time Complexity	Best Use Case
Phase 2 Dijkstra	Linear Set	$O(V^2)$	Dense graphs, small datasets
Phase 3 Dijkstra	Min-Heap	$O((V+E)\log V)$	Sparse graphs, large datasets
Phase 3 Bellman-Ford	Edge List	$O(V \cdot E)$	Graphs with negative weights

Dense Graphs

$$E \approx V^2$$

Fully connected networks (e.g., flight routes)

Sparse Graphs

$$E \approx V$$

Road networks (4-6 connections per node)

Performance Gain

$$O(V^2) \rightarrow O(V \log V)$$

Significant improvement for sparse graphs

Key Insight

For delivery route optimization (sparse graphs), the Min-Heap implementation provides **exponential performance improvement** over the linear approach, while Bellman-Ford enables handling of energy rebates (negative weights) at the cost of higher computational complexity.

Web Application Demo

Scan to Access



Scan with your mobile device
algorithm-project-silk.vercel.app

Key Features

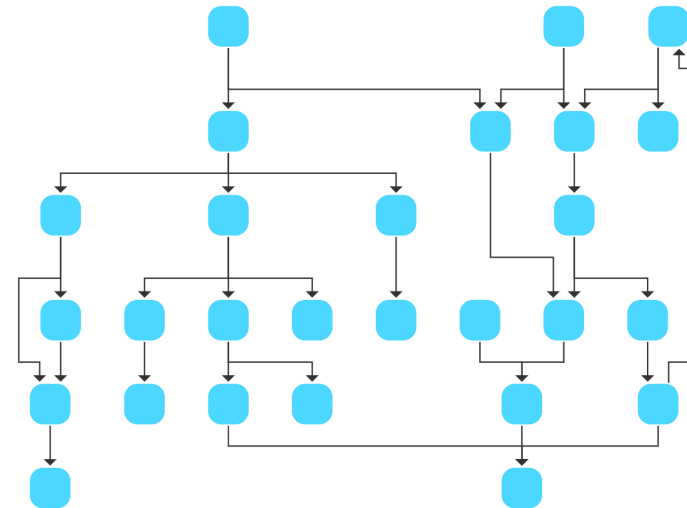
-  **Interactive Map Visualizer** with real-time algorithm execution
-  Multiple algorithm implementations
-  Performance benchmarking with detailed metrics
-  Side-by-side algorithm comparison

Dijkstra

Bellman-Ford

Min-Heap

Application Interface

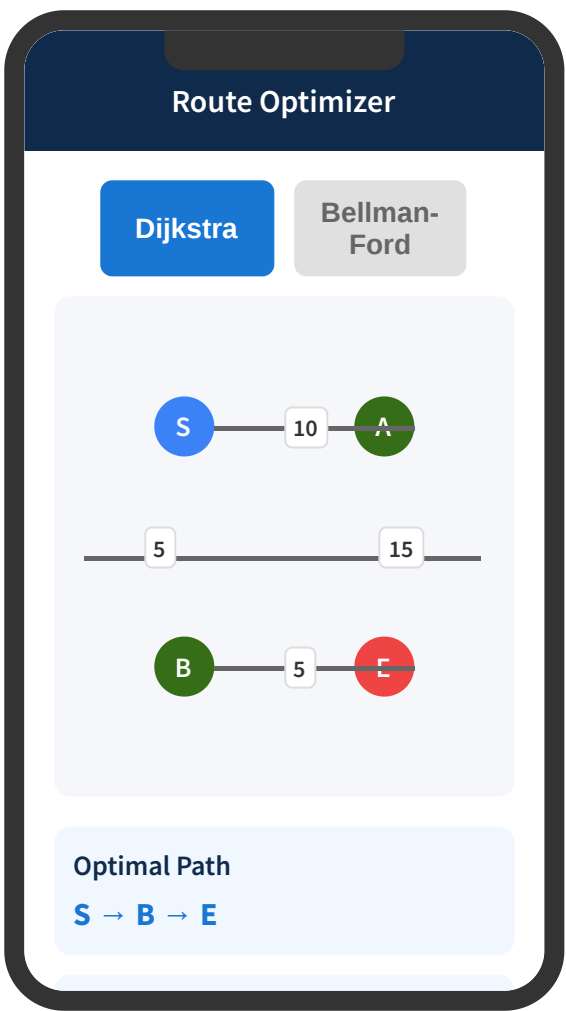


Test Case: Simple Map

Test Results

- ✓ **Basic Functionality** Simple graph structure with clear path differentiation
- ✓ **Algorithm Correctness** Both Dijkstra and Bellman-Ford work correctly
- 📏 **Performance** Min-Heap shows **2.5x faster** execution than linear search
- 📌 **Key Insight** No negative weights, so greedy approach succeeds

Mobile View



Mobile View: Simple Map Test

Test Case: Negative Weight Failure

⚠️ Test Results

❌ **Dijkstra Failure** Greedy approach fails with negative edge weights

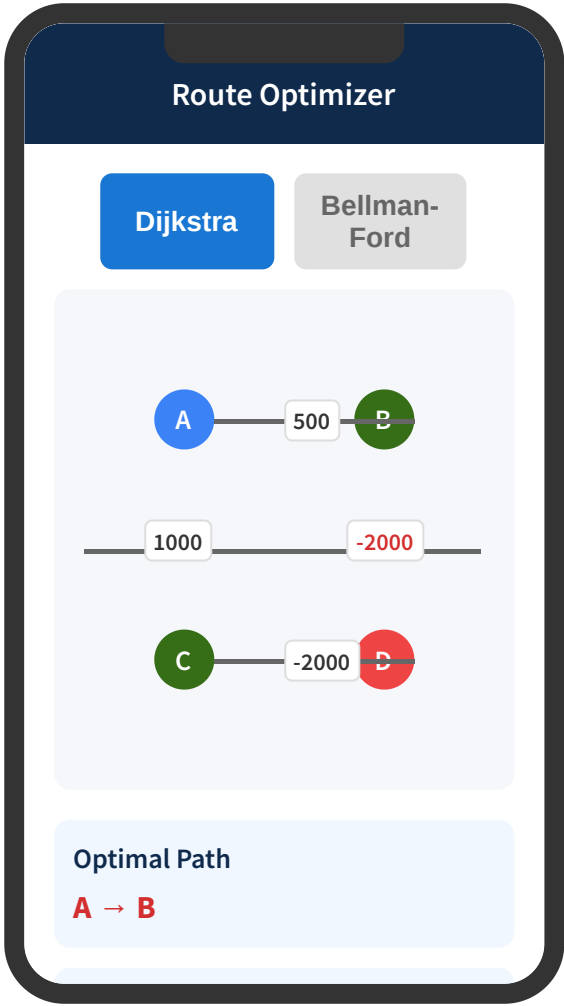
✅ **Bellman-Ford Success** Correctly handles negative weights

💡 **Key Insight** Dijkstra's assumption: **once visited, path is final**

↔️ Energy rebates require **dynamic programming** approach

Algorithm	Path	Cost	Result
Dijkstra	A → B	500	Incorrect
Bellman-Ford	A → C → B	-1000	Correct

📱 Mobile View



Mobile View: Negative Weight Test

Test Case: Complex Negative Cycle

⚠️ Test Results

🔄 **Negative Cycle** Creates infinite energy (undefined shortest path)

✅ **Bellman-Ford Detection** Identifies and reports the negative cycle

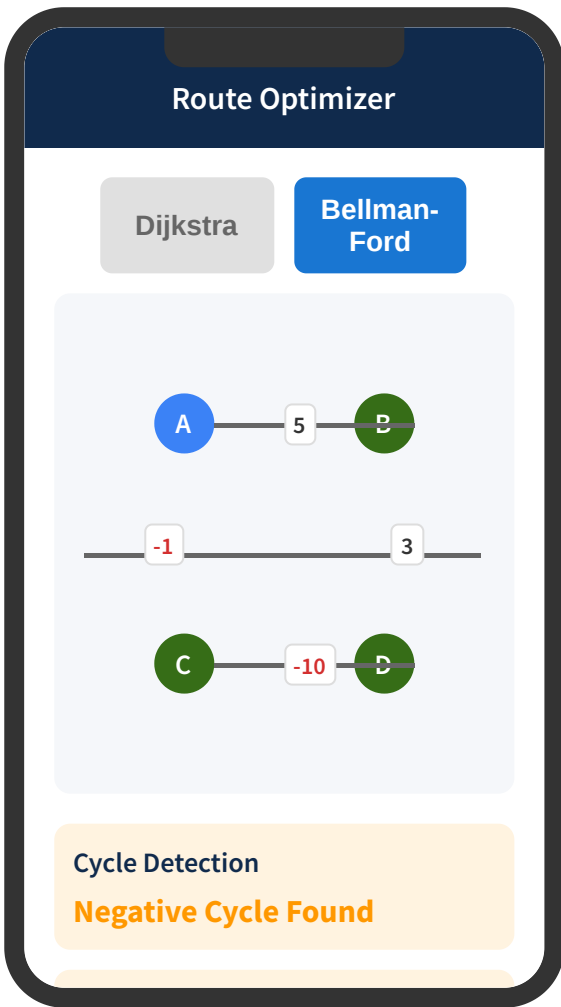
⌚ **Dijkstra Limitation** Fails silently or produces incorrect results

📈 **$|V|$ iterations** required to detect negative cycles

📌 Negative Cycle Impact

In energy grids, negative cycles represent **impossible physical scenarios** where infinite energy can be generated, violating thermodynamics.

📱 Mobile View



Mobile View: Negative Cycle Detection

Test Case: The Trap Map

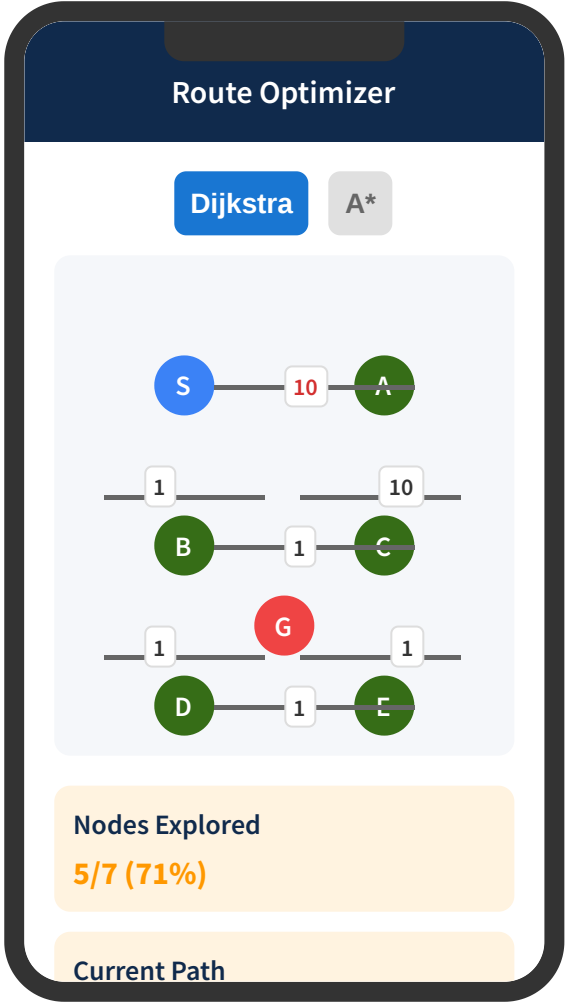
⚠️ Test Results

- 🔍 **Exploration Inefficiency** Dijkstra explores entire irrelevant subgraph
- 📉 **Low-Weight Trap** Leads to long path or dead end
- 📈 **Performance Impact** **5x more nodes visited** than necessary
- 💡 **Solution** Heuristic search (A*) needed for large-scale maps

📌 Trap Mechanism

The algorithm follows a **"locally optimal" path** that appears promising initially but leads away from the goal, forcing extensive backtracking.

📱 Mobile View



Mobile View: Trap Map Test

Test Case: Dangerous Negative Map

⚠️ Test Results

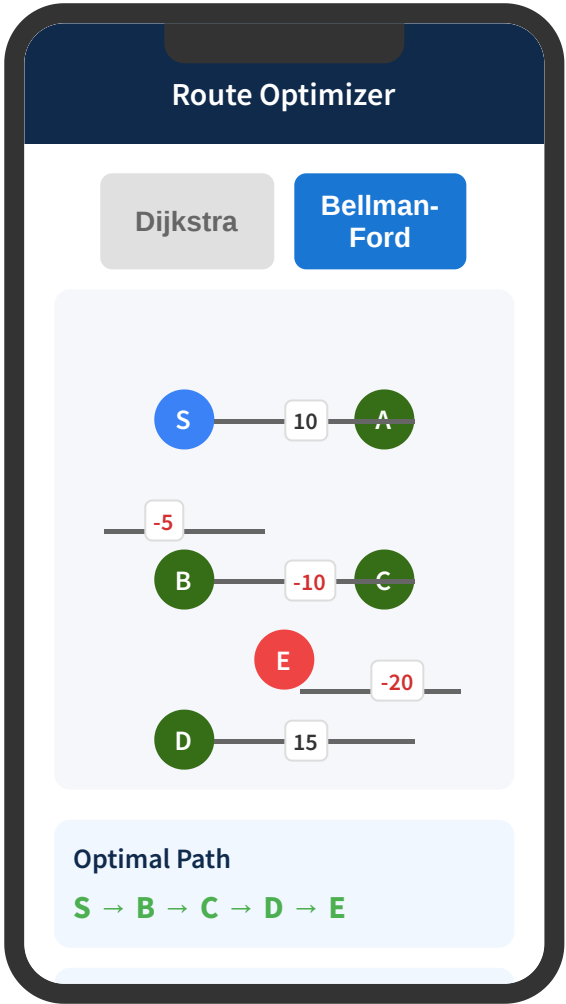
Algorithm	Cost	Time	Result
Dijkstra (Set)	-179	1.8 ms	Incorrect
Dijkstra (Heap)	-166	1.3 ms	Incorrect
Bellman-Ford	-227	9.7 ms	Correct

⚠️ **Dijkstra Failure** Different implementations produce different incorrect results

✅ **Bellman-Ford Success** Finds true optimal path with **-227 cost**

⚖️ **Trade-off** **7.5× slower** but necessary for energy grids with rebates

📱 Mobile View

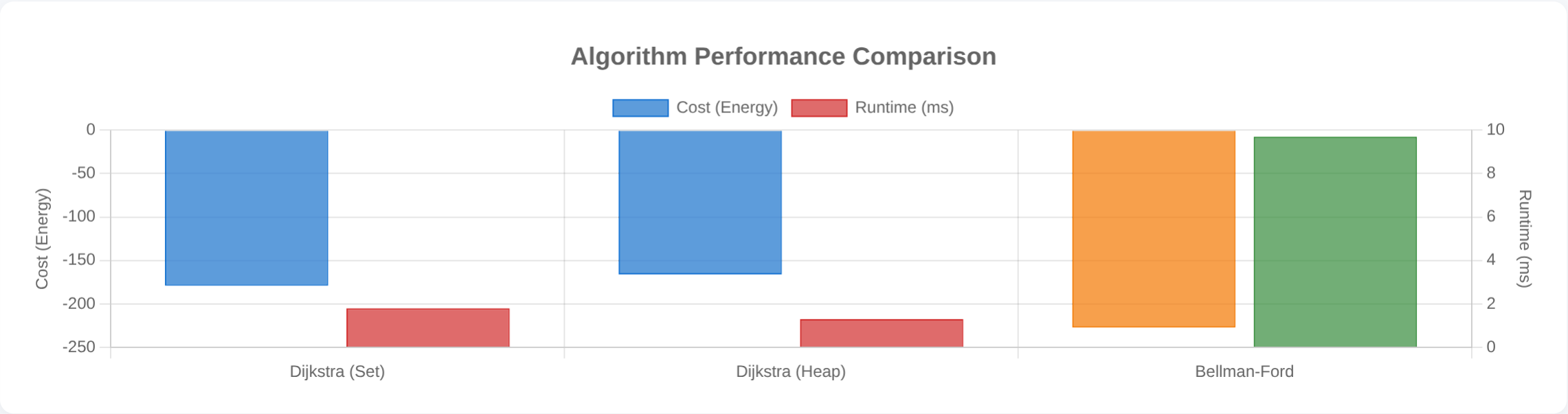


Mobile View: Dangerous Negative Map Test

Empirical Testing: The 'Dangerous Map' Case

Testing with negative edge weights reveals critical differences between algorithms

Algorithm	Cost (Energy)	Runtime	Result
Dijkstra (Set)	-179	1.8 ms	Incorrect
Dijkstra (Heap)	-166	1.3 ms	Incorrect
Bellman-Ford	-227	9.7 ms	Correct



Key Finding

Dijkstra's greedy approach fails with negative weights, producing suboptimal paths. Bellman-Ford, though **7.5x slower**, correctly identifies the optimal path by allowing path updates after initial visits.

Analysis of Failure and Success

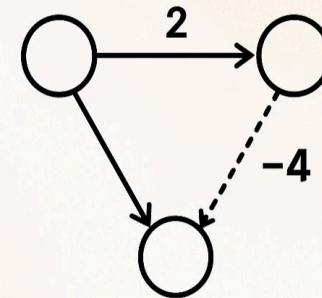
❗ Why Dijkstra Failed

- ❌ **Greedy assumption**: Once visited, path is final
- ❌ Negative edges **violate** this assumption
- ❌ Algorithm "locks" nodes too early
- ❌ Cannot improve paths after visiting nodes

✅ Why Bellman-Ford Succeeded

- ↻ **Dynamic Programming** approach
- ↻ Relaxes every edge $|V| - 1$ times
- ↻ Allows **path updates** after initial visits
- ↻ Finds true optimal path with negative weights

Bellman-Ford Algorithm



💡 Key Insight

Energy Grids require **correctness over speed**. Bellman-Ford's 7.5× slower runtime is necessary to accurately model energy rebates.

Test Case: Dangerous Negative Map

⚠️ Test Results

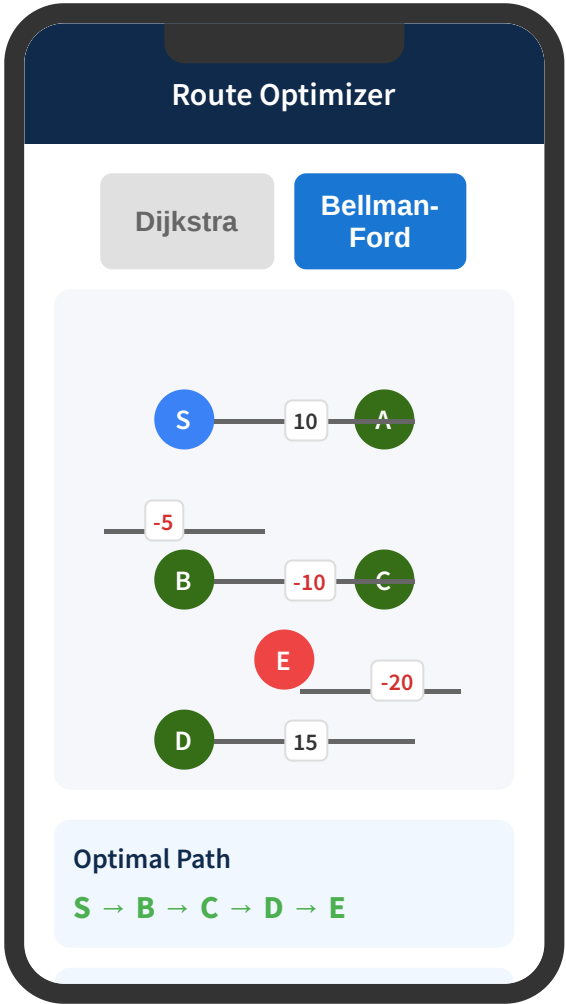
Algorithm	Cost	Time	Result
Dijkstra (Set)	-179	1.8 ms	Incorrect
Dijkstra (Heap)	-166	1.3 ms	Incorrect
Bellman-Ford	-227	9.7 ms	Correct

⚠️ **Dijkstra Failure** Different implementations produce different incorrect results

✅ **Bellman-Ford Success** Finds true optimal path with **-227 cost**

⚖️ **Trade-off** **7.5× slower** but necessary for energy grids with rebates

📱 Mobile View



Mobile View: Dangerous Negative Map Test

Advanced Theoretical Context

Algorithm Classification

-  **Greedy** Dijkstra makes locally optimal choices
-  **Dynamic** Bellman-Ford revisits and updates paths

The TSP Problem

-  Multiple steps: $A \rightarrow B \rightarrow C \rightarrow A$
-  **NP-Hard** Exponential complexity $O(n^2 \cdot 2^n)$
-  50 steps = **longer than universe age**

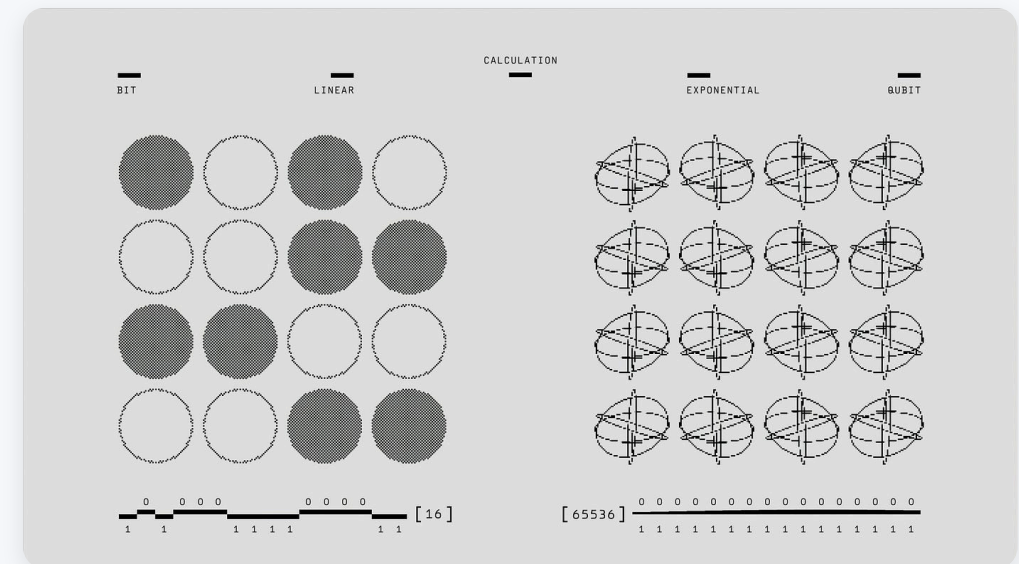
Computational Reality

$$50! \approx 3.04 \times 10^{64}$$

Calculation time > age of universe

Complexity Classes

-  **Class P** Polynomial time (Dijkstra, Bellman-Ford)
-  **Class NP** Verifiable in polynomial time
-  **NP-Hard** At least as hard as NP problems



Future Paradigms

-  **Quantum** Grover's Algorithm: $O(\sqrt{N})$
-  Biological computing: Protein folding
-  Neuromorphic architectures

Theoretical Limits

Classical computing bound by **Church-Turing Thesis**. Future paradigms may shift computational boundaries, potentially solving currently intractable problems.

Beyond Current Limits: The TSP Problem and Paradigm Shifts

Why TSP Remains Unsolved

-  **NP-Hard** Exponential complexity $O(n^2 \cdot 2^n)$
-  **Bremermann's Limit** Maximum computational speed
-  **Turing Machine** Classical computing model
-  Today's physics cannot solve TSP efficiently for large N

Paradigm Shifts in Computing

-  We thought Earth was flat, space filled with **Aether**, time constant
-  Why assume **NP-Hard** is the final limit?
-  Lord Kelvin (1800s): "Nothing new to discover in physics"
-  **Turing Machine** may be our "Aether"

Breaking the Limits

-  **Quantum** Grover's Algorithm - first crack in the wall
-  **CTCs** Closed Timelike Curves - loops in time
-  **Hypercomputation** Beyond 0s and 1s
-  **Dark Matter** 95% of universe we don't understand

The Philosophical Argument

-  **Current Reality:** Classical computers, TSP is intractable
-  **Future Reality:** "Impossible" is temporary
-  General Relativity replaced Classical Mechanics
-  Future paradigm may render **NP-Hardness irrelevant**

Final Thought

Until that paradigm shift arrives, we must **approximate** — but we should never stop questioning what we consider impossible.

Two Perspectives on Computational Limits



Current Reality

The Engineer's View

-  Classical silicon computers
-  Current mathematical frameworks
-  TSP is **Intractable (NP-Hard)**
-  Solution: **Bellman-Ford** or **Genetic Algorithms**



Future Reality

The Scientist's View

-  "Impossible" is **temporary**
-  General Relativity → Classical Mechanics
-  Future paradigm: **Quantum Gravity, Biological Computing**
-  May render **NP-Hardness irrelevant**

"Until that day comes, we approximate."